GitHub Actions — Category 9: Observability, Debugging & Operations

Complete Interview Deep-Dive for SRE / Platform / DevOps Engineers

> **Prerequisites:** Categories 1–8 assumed. Familiarity with GitHub API and basic shell scripting assumed. **Format per topic:** What → Why → How Internally → Key Concepts → Interview Answers → Full YAML/Config → Gotchas ⚠ → Connections

---

Table of Contents

- 9.1 Workflow Logs — Structured Logging, Workflow Commands
- 9.2 Debugging with tmate and act (Local Runner)
- 9.3 Status Badges and Workflow Observability
- 9.4 GitHub Actions API — Querying Runs, Triggering via API/CLI
- 9.5 Rate Limits, Usage Limits, and Billing Visibility
- 9.6 Audit Logs for GitHub Actions at Org Level
- Cross-Topic Interview Questions
- Quick Reference Cheat Sheet

9.1 Workflow Logs — Structured Logging, Workflow Commands

What It Is

GitHub Actions provides a set of special log commands — called **workflow commands** — that control how output appears in the Actions UI. These go beyond plain `echo` statements: they create collapsible log groups, annotate code with inline warnings and errors, mask sensitive values, set environment variables mid-job, and write to the job summary panel.

Jenkins equivalent: Jenkins' `echo`, `ansiColor`, and `catchError` steps, plus the Build Failure Analyzer plugin for annotations. The difference: GitHub's workflow commands are simple `::keyword::` syntax written to stdout, requiring no plugins, and they integrate directly into the GitHub PR review UI as inline code annotations.

The Workflow Command Protocol

All workflow commands follow a single format written to stdout:

```
::workflow-command parameter1=value1,parameter2=value2::message
```

GitHub's runner intercepts any line matching this pattern and processes it as a command — the line does NOT appear as regular log output (unless it fails to parse).

```
# The runner reads stdout line by line
# Lines matching ::command:: format are PROCESSED, not printed
# All other lines are printed as normal log output

echo "This is a regular log line — appears in logs"
echo "::notice::This is a notice command — processed as annotation"
echo "::debug::This appears only if ACTIONS_STEP_DEBUG=true"
```

Log Grouping — `::group::` and `::endgroup::`

```
steps:
  - name: Build with grouped output
    run: |
      # Creates a collapsible section in the Actions UI
      echo "::group::Installing dependencies"
      npm ci
      echo "::endgroup::"

      echo "::group::Running tests"
      npm test 2>&1
      echo "::endgroup::"

      echo "::group::Building artifacts"
      npm run build
      ls -la dist/
      echo "::endgroup::"

      # Groups can be nested conceptually but NOT syntactically
      # A new ::group:: before ::endgroup:: closes the previous one

  - name: Structured diagnostic output
    if: failure()
    run: |
      echo "::group::Environment diagnosis"
      echo "Node version: $(node --version)"
      echo "npm version: $(npm --version)"
      echo "Working directory: $(pwd)"
      echo "Disk usage:"
      df -h
      echo "Memory:"
      free -m
      echo "::endgroup::"

      echo "::group::Recent logs"
```

```
      tail -100 /var/log/app.log 2>/dev/null || echo "No app log found"
      echo "::endgroup::"
```

Annotations — `::notice::`, `::warning::`, `::error::`

Annotations appear in three places:

1. The workflow run log (inline with the step output)
2. The PR "Checks" tab as annotations linked to specific files/lines
3. The repository's "Security" tab (for `::error::` from security tools)

```
steps:
  - name: Demonstrate all annotation levels
    run: |
      # Notice: informational (blue highlight in UI)
      echo "::notice::Deployment target: production-us-east-1"
      echo "::notice file=src/config.ts,line=42::Configuration loaded from
environment"

      # Warning: non-blocking (yellow highlight)
      echo "::warning::Docker image size is 1.2GB -- consider multi-stage
build"
      echo "::warning file=Dockerfile,line=15,col=1::Large COPY command
may slow builds"

      # Error: blocking-level (red highlight, marks step as failed if
combined with exit 1)
      echo "::error::Database migration 20240315 failed validation"
      echo "::error file=migrations/20240315.sql,line=47::Invalid FK
constraint reference"

      # Title parameter for better UI display
      echo "::warning title=Deprecation Warning::API v1 endpoint /users
will be removed in v3.0"
      echo "::error title=Test Failure::TestPaymentProcessor.TestRefund
failed: expected 200 got 422"

  - name: Annotations from test results (SARIF pattern)
    run: |
      # Parse test output and emit annotations
      npm test --reporter=json 2>&1 | python3 -c "
      import json, sys

      data = json.load(sys.stdin)
      for test in data.get('testResults', []):
          for failure in test.get('failureMessages', []):
              # Extract file/line from stack trace if available
              file_path = test.get('testFilePath', '').replace('$(pwd)/',
'')
              print(f'::error file={file_path},title=Test Failure::
```

```
{failure[:200]}')
      "
```

Secret Masking — `::add-mask::`

```
steps:
  - name: Fetch external secret and mask it
    run: |
      # Secrets from vault or other sources aren't auto-masked
      # You MUST mask them manually before any logging

      # Fetch from Vault
      VAULT_SECRET=$(vault kv get -field=password secret/myapp/db)

      # Mask IMMEDIATELY before any other command that might log it
      echo "::add-mask::${VAULT_SECRET}"

      # Now safe to use -- any appearance in logs is replaced with ***
      export DATABASE_PASSWORD="${VAULT_SECRET}"
      echo "Database password configured: ${DATABASE_PASSWORD}"
      # Outputs: "Database password configured: ***"

  - name: Mask dynamic values
    run: |
      # Generate and mask a temporary token
      TEMP_TOKEN=$(generate-token.sh)
      echo "::add-mask::${TEMP_TOKEN}"

      # Mask must be set BEFORE any echo/print that could log the value
      # Setting mask AFTER a print does NOT retroactively mask earlier
output
```

Debug Logging — `::debug::`

```
steps:
  - name: Debug output (only visible when debug mode enabled)
    run: |
      echo "::debug::Starting service detection"
      echo "::debug::Changed files: $(git diff --name-only HEAD~1)"
      echo "::debug::Environment: $RUNNER_OS / $RUNNER_ARCH"

      # Regular log -- always visible
      echo "Processing 42 services"

      echo "::debug::Matrix computation complete"
      echo "::debug::$(jq . services.json)"  # Full JSON only in debug
mode
```

```
# Enable debug logging:
# Option 1: Repository secret ACTIONS_STEP_DEBUG = true
# Option 2: Re-run job with "Enable debug logging" checkbox
# Option 3: ACTIONS_RUNNER_DEBUG = true (also enables runner diagnostics)
```

Step Summary — `$GITHUB_STEP_SUMMARY`

The step summary file allows you to write rich Markdown content that appears in a dedicated "Summary" tab of the workflow run — NOT in the raw logs.

```
steps:
  - name: Generate deployment summary
    run: |
      cat >> $GITHUB_STEP_SUMMARY << 'EOF'
      ## Deployment Summary

      | Service | Version | Environment | Status |
      |---------|---------|-------------|--------|
      | auth-api | sha-abc123 | production | ✅ Deployed |
      | payment-api | sha-abc123 | production | ✅ Deployed |
      | inventory | sha-abc123 | production | ⏳ Pending |

      ### Deployment Details
      - **Triggered by:** @dev-username
      - **Commit:** [`abc1234`](https://github.com/my-org/my-
repo/commit/abc1234)
      - **Duration:** 4m 32s
      - **Rollout strategy:** Canary (10% → 100%)

      ### Next Steps
      - [ ] Monitor error rates for 30 minutes
      - [ ] Verify Datadog dashboards are healthy
      - [ ] Close the deploy Jira ticket
      EOF

  - name: Test results summary
    run: |
      # Write test results to summary (Markdown tables work great here)
      PASS=$(grep -c "PASS" test-results.txt || echo 0)
      FAIL=$(grep -c "FAIL" test-results.txt || echo 0)
      SKIP=$(grep -c "SKIP" test-results.txt || echo 0)
      TOTAL=$((PASS + FAIL + SKIP))

      cat >> $GITHUB_STEP_SUMMARY << EOF
      ## Test Results

      | Metric | Count |
      |--------|-------|
      | ✅ Passed | ${PASS} |
```

```
        | ❌ Failed | ${FAIL} |
        | ⏭️ Skipped | ${SKIP} |
        | **Total** | **${TOTAL}** |

        $(if [ "$FAIL" -gt 0 ]; then
          echo "### Failed Tests"
          grep "FAIL" test-results.txt | head -20 | while read LINE; do
            echo "- \`${LINE}\`"
          done
        fi)
        EOF

    - name: Coverage report to summary
      run: |
        # Append coverage data to summary (multiple steps can append)
        COVERAGE=$(go tool cover -func=coverage.out | tail -1 | awk '{print
$3}')
        echo "**Test Coverage:** ${COVERAGE}" >> $GITHUB_STEP_SUMMARY
```

Environment Variables and Path — `$GITHUB_ENV` and `$GITHUB_PATH`

```
steps:
  - name: Set variables for subsequent steps
    run: |
      # Set environment variable (available to ALL subsequent steps in
job)
      VERSION=$(cat VERSION)
      echo "APP_VERSION=${VERSION}" >> $GITHUB_ENV
      echo "DEPLOY_TIME=$(date -u +%Y-%m-%dT%H:%M:%SZ)" >> $GITHUB_ENV

      # Multiline value:
      CHANGELOG=$(cat CHANGELOG.md | head -50)
      echo "CHANGELOG<<EOF" >> $GITHUB_ENV
      echo "$CHANGELOG" >> $GITHUB_ENV
      echo "EOF" >> $GITHUB_ENV

  - name: Use variable set by previous step
    run: |
      # APP_VERSION is now available without re-running the command
      echo "Deploying version: $APP_VERSION"
      echo "Deploy time: $DEPLOY_TIME"

  - name: Add to PATH
    run: |
      # Add a directory to PATH for subsequent steps
      echo "/home/runner/custom-tools/bin" >> $GITHUB_PATH
      echo "/opt/my-org/scripts" >> $GITHUB_PATH

  - name: Use tool from modified PATH
    run: |
```

```
        # my-tool is now on PATH
        my-tool --version
```

Complete Logging Reference — All Workflow Commands

```
# ── Annotations ──────────────────────────────────────────────
::debug::message
::notice::message
::notice
file=app.js,line=1,endLine=5,col=1,endColumn=10,title=Title::message
::warning::message
::warning
file=path,line=N,endLine=N,col=N,endColumn=N,title=Title::message
::error::message
::error file=path,line=N,endLine=N,col=N,endColumn=N,title=Title::message

# ── Masking ──────────────────────────────────────────────────
::add-mask::value                    # Mask value in all subsequent log
output

# ── Grouping ─────────────────────────────────────────────────
::group::Group title                 # Start collapsible group
::endgroup::                         # End collapsible group

# ── Stop/start log processing ────────────────────────────────
::stop-commands::token               # Stop processing commands until...
::token::                            # ...this token is seen
# (Useful when outputting content that looks like commands)

# ── Output (deprecated -- use GITHUB_OUTPUT file instead) ────
# OLD (don't use): ::set-output name=key::value
# NEW: echo "key=value" >> $GITHUB_OUTPUT

# ── Environment (deprecated -- use GITHUB_ENV file instead) ──
# OLD (don't use): ::set-env name=KEY::value
# NEW: echo "KEY=value" >> $GITHUB_ENV
```

Interview Answer (30-45 seconds)

> "GitHub Actions workflow commands are a protocol embedded in stdout — lines matching `::command::` syntax are intercepted by the runner and processed rather than printed. The most important ones: `::group::` and `::endgroup::` create collapsible log sections in the UI — essential for long build outputs. `::error::`, `::warning::`, and `::notice::` with optional `file=` and `line=` parameters create inline code annotations that appear in the PR Checks tab and link directly to the relevant source line. `::add-mask::` registers a value as sensitive so it's replaced with `***` in all subsequent log output — critical for secrets fetched from external vaults that aren't auto-masked by GitHub. `$GITHUB_STEP_SUMMARY` is the most impactful for developer experience: it renders

> Markdown in the workflow run's Summary tab, perfect for test result tables, deployment reports, and coverage summaries that would be unreadable in raw log output."

Gotchas ⚠

- **`::add-mask::` must be called BEFORE the value appears in any log output.** Setting a mask after an echo doesn't retroactively redact previous output. The value is already in the log. Order of operations matters critically.
- **`::set-output::` and `::set-env::` are permanently deprecated.** Using them triggers security warnings. Always use `echo "key=value" >> $GITHUB_OUTPUT` and `echo "KEY=value" >> $GITHUB_ENV` respectively. The deprecation reason: the old commands were vulnerable to command injection via untrusted content.
- **`::stop-commands::` is the escape hatch for outputting content containing `::`.** If your build output contains strings that look like workflow commands (e.g., from certain testing frameworks), wrap the output with `::stop-commands::TOKEN` and `::TOKEN::` to prevent accidental command execution.
- **Step summaries are limited to 1MB per step and 20MB per workflow run.** Writing massive logs to `$GITHUB_STEP_SUMMARY` (e.g., full test output) will be silently truncated. Write summaries, not raw logs.
- **Annotations are per-workflow-run, not persisted across runs.** If a run produces 50 warning annotations and you re-run the job, the 50 warnings are replaced by the new run's annotations, not accumulated.
- **File path in annotations must be relative to the workspace root.** `file=/home/runner/work/repo/src/handler.go` won't link correctly. Use `file=src/handler.go`.

9.2 Debugging with tmate and act (Local Runner)

What It Is

Two distinct debugging approaches: **tmate** is a tool that pauses a running GitHub Actions job and provides SSH/browser access directly into the runner's shell — live interactive debugging of a running CI environment. **act** is a local CLI tool that runs GitHub Actions workflows on your local machine using Docker — local iteration without pushing commits.

Jenkins equivalent: Jenkins has no equivalent to tmate — you'd SSH to the agent manually if you knew its address. For local testing, Jenkins declarative pipeline linting (`jenkins-linter`) is the closest equivalent to `act`. GitHub Actions' debugging story, particularly tmate, is significantly more powerful.

---

tmate — Interactive Live Debugging

How tmate Works

```
Normal workflow execution:
  Runner executes steps sequentially
  Logs streamed to GitHub
  Job completes -> runner cleaned up

With tmate:
  Steps execute until tmate step is reached
  tmate opens an SSH session
  Runner pauses -- waits for you to connect
  YOU GET A SHELL on the actual runner
  Full access: filesystem, environment, secrets (masked), tools
  You run commands, inspect state, fix issues
  Type exit to end session -> workflow continues or fails
```

Basic tmate Usage

```yaml
name: Debug Workflow

on: [push, workflow_dispatch]

jobs:
  debug:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - name: Run tests (may fail)
        id: tests
        run: npm test
        continue-on-error: true    # Don't stop on failure -- tmate still
connects

      # Insert tmate AFTER the failing step to inspect state
      - name: Setup tmate debug session
        uses: mxschmitt/action-tmate@v3
        if: failure()    # Only debug when something fails
        # OR: always() to always get a shell
        # OR: no condition to always stop here
        with:
          limit-access-to-actor: true    # Only the PR author can SSH in

      # Steps after tmate continue when you exit the session
      - name: Post-debug step
        run: echo "Continuing after debug session"
```

Advanced tmate Patterns

```yaml
      # ── Pattern 1: Debug on explicit failure with timeout ────────
      - name: Debug session (auto-exits after 15 min)
        uses: mxschmitt/action-tmate@v3
        if: failure()
        with:
          limit-access-to-actor: true
          timeout-minutes: 15    # Auto-exit to avoid orphaned runners

      # ── Pattern 2: Conditional debug via workflow input ──────────
      # Allow enabling debug mode without editing the workflow file
      - name: Debug session if requested
        if: ${{ github.event_name == 'workflow_dispatch' &&
inputs.debug_enabled }}
        uses: mxschmitt/action-tmate@v3
        with:
          limit-access-to-actor: true

      # ── Pattern 3: Debug in matrix builds ────────────────────────
      # Debug only the failing matrix combination
      - name: Debug specific matrix failure
        if: failure() && matrix.os == 'windows-latest'
        uses: mxschmitt/action-tmate@v3
        with:
          limit-access-to-actor: true
```

```yaml
# workflow with debug input flag
name: CI with Debug Mode

on:
  push:
    branches: [main]
  workflow_dispatch:
    inputs:
      debug_enabled:
        type: boolean
        description: 'Enable tmate debug session on failure'
        required: false
        default: false

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - name: Run tests
        id: test
        run: npm test
        continue-on-error: true

      - name: Debug session
```

```
        uses: mxschmitt/action-tmate@v3
        if: |
          (failure() || steps.test.outcome == 'failure') &&
          github.event.inputs.debug_enabled == 'true'
        with:
          limit-access-to-actor: true
          timeout-minutes: 30
```

What You Can Do in a tmate Session

```
# Once connected via SSH or browser link:

# Inspect the filesystem
ls -la /home/runner/work/my-repo/my-repo/
cat /home/runner/work/my-repo/my-repo/package.json

# Check environment variables (GitHub masks secrets)
env | grep GITHUB_
env | grep NODE_

# Check what the failing test actually saw
cd /home/runner/work/my-repo/my-repo
cat test-results.xml
npm test -- --verbose --testNamePattern="FailingTest"

# Inspect installed tools
which node && node --version
which docker && docker --version
docker ps -a    # Running service containers

# Check disk space (common failure cause)
df -h
du -sh node_modules/

# Try fixes interactively
npm install missing-package
node -e "require('./src/failing-module')"

# Exit when done -- workflow continues from where it paused
exit
```

---

act — Local Workflow Execution

What act Does

`act` uses Docker to simulate the GitHub Actions runner environment locally. It reads your workflow YAML and executes jobs in Docker containers that closely mirror GitHub-hosted runners.

```
Without act:
  Edit workflow -> git push -> wait 3-10 minutes for runner -> see error
  Fix -> push -> wait again -> iterate

With act:
  Edit workflow -> act --job build -> see error in 10 seconds
  Fix -> act --job build -> iterate locally
  Much faster feedback loop for workflow development
```

Installing and Configuring act

```
# Install act
# macOS
brew install act

# Linux
curl -s https://raw.githubusercontent.com/nektos/act/master/install.sh |
sudo bash

# Windows (via GitHub CLI extension)
gh extension install nektos/gh-act

# ── Configuration ──────────────────────────────────────────────────
# act reads .actrc in your project root or ~/.actrc

# .actrc (project root)
# Default runner image -- micro is small/fast but less tool-complete
# medium has more tools, full is closest to GitHub-hosted but large
-P ubuntu-latest=catthehacker/ubuntu:act-22.04
-P ubuntu-22.04=catthehacker/ubuntu:act-22.04
-P ubuntu-20.04=catthehacker/ubuntu:act-20.04
-P windows-latest=-self-hosted   # act cannot run Windows containers on
Linux

# Secrets file (never commit this)
--secret-file .secrets
```

```
# .secrets file (add to .gitignore)
GITHUB_TOKEN=ghp_your_token_here
AWS_ACCESS_KEY_ID=AKIA...
AWS_SECRET_ACCESS_KEY=...
NPM_TOKEN=...
```

Common act Commands

```
# List all available jobs
act --list

# Run the default event (push)
act

# Run a specific job
act --job test
act --job build
act --job deploy

# Run with a specific event
act pull_request
act push
act workflow_dispatch

# Run with workflow_dispatch inputs
act workflow_dispatch \
  --input environment=staging \
  --input dry_run=true

# Run with secrets
act --secret-file .secrets
act --secret MY_SECRET=value

# Run with environment variables
act --env NODE_ENV=test

# Run a specific event with a custom payload
act push --eventpath custom-push-event.json

# Dry run -- show what would execute without running
act --dryrun

# Use a specific runner image
act --platform ubuntu-latest=catthehacker/ubuntu:act-22.04

# Verbose mode (shows all command output)
act --verbose

# Watch mode -- re-run on workflow file change
act --watch

# Run only specific steps (useful for skipping setup)
# act doesn't support step-level targeting, but you can use job deps
```

act Limitations and Workarounds

```
LIMITATION 1: Services containers are not fully supported
  Docker-in-Docker situations break on some platforms
```

```
  Workaround: Use docker-compose up manually and point act at the running
services

LIMITATION 2: GitHub-specific contexts differ
  github.actor, github.sha, github.event.* have mock values
  Workaround: Use --eventpath to supply a realistic event JSON
  # Example realistic push event:
  cat > .act-events/push.json << 'EOF'
  {
    "ref": "refs/heads/main",
    "before": "abc123",
    "after": "def456",
    "repository": {"full_name": "my-org/my-repo", "private": true},
    "pusher": {"name": "dev-user"},
    "commits": [{"id": "def456", "message": "Fix: payment refund logic"}]
  }
  EOF
  act push --eventpath .act-events/push.json

LIMITATION 3: OIDC tokens cannot be generated locally
  Workaround: Mock the cloud provider calls or use static credentials in
.secrets

LIMITATION 4: actions/cache doesn't work with act by default
  Workaround: Use act's built-in artifact server or disable cache steps
  act --artifact-server-path /tmp/act-artifacts

LIMITATION 5: Large runner images (full) are very large (>10GB)
  Workaround: Use medium images and install missing tools in a pre-step

LIMITATION 6: macOS and Windows runners cannot be simulated on Linux
  Workaround: Test cross-platform logic on GitHub-hosted runners
```

act for Rapid Workflow Development

```
# Typical act development workflow:

# 1. Start with smallest usable image
act --job lint --platform ubuntu-latest=catthehacker/ubuntu:act-22.04

# 2. Mock only what you need
cat > .secrets << 'EOF'
GITHUB_TOKEN=mock_token
NPM_TOKEN=real_token_for_private_packages
EOF

# 3. Iterate fast
# Edit .github/workflows/ci.yml
# act --job lint
# Edit -> act -> edit -> act (seconds per iteration, not minutes)
```

```
# 4. Once working locally, push and validate on GitHub
git push origin feature/fix-workflow
# (GitHub-hosted run confirms real secrets, OIDC, caching work)
```

Interview Answer (30-45 seconds)

> "There are two main debugging tools. tmate gives you an interactive SSH shell directly inside a running GitHub Actions job. You add the `mxschmitt/action-tmate` step, optionally conditional on `failure()`, and when the runner hits that step it pauses and prints an SSH command. You connect, inspect the exact filesystem state, environment, and tooling at the point of failure, run commands interactively to diagnose or test fixes, then type exit to continue. Always set `limit-access-to-actor: true` and a `timeout-minutes` to prevent abandoned runner sessions. Act runs workflows locally using Docker, giving fast iteration feedback without pushing commits. It reads your workflow YAML and executes jobs in containers that approximate GitHub-hosted runners. The key limitation: OIDC doesn't work locally, services containers have reduced support, and GitHub-specific contexts have mock values. I use act for workflow syntax and logic development, tmate for diagnosing mysterious failures that only appear in the real CI environment."

Gotchas ⚠

- **tmate exposes all unmasked environment variables to the connected user.** Even `limit-access-to-actor: true` only restricts who can connect — once connected, that user sees the runner's full environment. `GITHUB_TOKEN` is visible; secrets are masked by `***` in outputs but MAY be readable via `/proc` on some systems. Never use tmate on production runners.
- **tmate sessions on public repos are a serious security risk.** Any collaborator who pushes to a public repo can trigger tmate sessions and SSH in. Only use tmate on private repos, or on workflow_dispatch with explicit input gating.
- **Forgetting `timeout-minutes` on tmate blocks the runner indefinitely.** A tmate session with no timeout and no connected user holds the runner job alive (and billing minutes accruing) until the workflow's maximum job timeout (6 hours). Always set a short timeout: `timeout-minutes: 15`.
- **act doesn't support all action types.** Docker-based actions work differently in act vs real GitHub Actions. JavaScript actions work best. Composite actions generally work. Some marketplace actions that depend on GitHub's internal infrastructure fail silently in act.
- **`act` caches Docker images after first pull.** If the workflow image changes on GitHub but your local act cache is stale, you'll be testing against old tooling. Run `docker pull catthehacker/ubuntu:act-22.04` periodically.
- **act doesn't enforce GitHub Actions billing or concurrency limits.** You can run 100 parallel jobs locally in act with no throttling — useful for testing, misleading for production capacity planning.

9.3 Status Badges and Workflow Observability

What It Is

Status badges are dynamically generated SVG images that reflect the current state of a workflow (passing, failing, or no status). They're embedded in README files to communicate CI health at a glance. Beyond badges, GitHub provides several native observability surfaces for workflows.

Status Badge URL Format

```
https://github.com/<owner>/<repo>/actions/workflows/<workflow-
file>.yml/badge.svg

Parameters:
  ?branch=<branch>    # Show status for a specific branch (default:
default branch)
  ?event=<event>      # Show status for a specific trigger event
```

```
<!-- In README.md -->

<!-- Basic badge -- shows default branch status -->
[![CI](https://github.com/my-org/my-
repo/actions/workflows/ci.yml/badge.svg)](https://github.com/my-org/my-
repo/actions/workflows/ci.yml)

<!-- Branch-specific badge -->
[![CI (main)](https://github.com/my-org/my-
repo/actions/workflows/ci.yml/badge.svg?branch=main)]
(https://github.com/my-org/my-repo/actions/workflows/ci.yml)

<!-- Event-specific badge -- only shows push event status -->
[![CI (push)](https://github.com/my-org/my-
repo/actions/workflows/ci.yml/badge.svg?event=push)]
(https://github.com/my-org/my-repo/actions/workflows/ci.yml)

<!-- Multiple badges for complex projects -->
[![Build](https://github.com/my-org/my-
repo/actions/workflows/build.yml/badge.svg)](https://github.com/my-org/my-
repo/actions/workflows/build.yml)
[![Test](https://github.com/my-org/my-
repo/actions/workflows/test.yml/badge.svg)](https://github.com/my-org/my-
repo/actions/workflows/test.yml)
[![Deploy](https://github.com/my-org/my-
repo/actions/workflows/deploy.yml/badge.svg?event=push)]
(https://github.com/my-org/my-repo/actions/workflows/deploy.yml)
[![Security](https://github.com/my-org/my-
repo/actions/workflows/security.yml/badge.svg)](https://github.com/my-
org/my-repo/actions/workflows/security.yml)
```

Badge Status Logic

```
Badge reflects: last completed run on the target branch for the target
event

Status -> Badge displays:
  All jobs succeeded -> passing (green)
  Any job failed     -> failing (red)
  Workflow cancelled -> cancelled (orange)
  No completed runs  -> no status (gray)

IMPORTANT: Badge caches aggressively (GitHub CDN caches ~5-10 minutes)
  A workflow that just failed may still show green for several minutes
  Don't use badges for real-time status monitoring -- use the API
```

Beyond Badges — GitHub's Native Observability Surfaces

1. Workflow Runs List

```
Repository -> Actions tab -> All Workflows

Shows:
  - All workflow runs across all workflows
  - Filter by: workflow, branch, actor, event, status
  - Run time, trigger, commit SHA, actor
  - Clickable to individual run details

URL format for filtered view:
  https://github.com/<owner>/<repo>/actions
    ?query=workflow%3ACI+branch%3Amain+is%3Afailure
```

2. Job and Step Timings

```
Individual run -> any job -> timing sidebar

Shows:
  - Queue time (time waiting for a runner)
  - Setup time (runner startup)
  - Per-step duration
  - Total job duration

Use this to identify:
  - Slow steps to optimize
  - Runner queue depth issues (long queue time)
  - Cache hit vs miss (setup-node with cache takes 2s vs 40s)
```

3. Deployment Environments Dashboard

```
Repository -> Environments

Shows:
  - All configured environments
  - Latest deployment state per environment
  - Deployment history
  - Who approved/rejected

API equivalent:
  GET /repos/:owner/:repo/deployments
  GET /repos/:owner/:repo/deployments/:id/statuses
```

Creating Metrics from Workflow Runs (Custom Observability)

```yaml
# Pattern: Emit metrics from workflows to your observability stack

name: CI with Metrics

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - name: Start timer
        id: timer
        run: echo "start=$(date +%s)" >> $GITHUB_OUTPUT

      - name: Run tests
        id: tests
        run: |
          npm test --reporter=json > test-results.json 2>&1
          PASS=$(jq '.numPassedTests' test-results.json)
          FAIL=$(jq '.numFailedTests' test-results.json)
          echo "passed=${PASS}" >> $GITHUB_OUTPUT
          echo "failed=${FAIL}" >> $GITHUB_OUTPUT
        continue-on-error: true

      - name: Emit metrics to Datadog
        if: always()
        run: |
          END=$(date +%s)
          START=${{ steps.timer.outputs.start }}
          DURATION=$((END - START))

          STATUS="${{ steps.tests.outcome }}"
          PASSED="${{ steps.tests.outputs.passed }}"
```

```
          FAILED="${{ steps.tests.outputs.failed }}"

          # Send metrics via Datadog API
          curl -s -X POST "https://api.datadoghq.com/api/v1/series" \
            -H "Content-Type: application/json" \
            -H "DD-API-KEY: ${{ secrets.DATADOG_API_KEY }}" \
            -d "{
              \"series\": [
                {
                  \"metric\": \"ci.test.duration\",
                  \"points\": [[$(date +%s), ${DURATION}]],
                  \"tags\": [
                    \"repo:${{ github.repository }}\",
                    \"branch:${{ github.ref_name }}\",
                    \"workflow:${{ github.workflow }}\",
                    \"status:${STATUS}\"
                  ]
                },
                {
                  \"metric\": \"ci.test.passed\",
                  \"points\": [[$(date +%s), ${PASSED:-0}]],
                  \"tags\": [\"repo:${{ github.repository }}\"]
                },
                {
                  \"metric\": \"ci.test.failed\",
                  \"points\": [[$(date +%s), ${FAILED:-0}]],
                  \"tags\": [\"repo:${{ github.repository }}\"]
                }
              ]
            }"

      - name: Emit metrics to Prometheus Pushgateway
        if: always()
        run: |
          DURATION=$(( $(date +%s) - ${{ steps.timer.outputs.start }} ))
          STATUS_VALUE=$([[ "${{ steps.tests.outcome }}" == "success" ]]
&& echo 1 || echo 0)

          cat << EOF | curl -s --data-binary @- \
            http://pushgateway.internal.my-
org.com:9091/metrics/job/github_actions/repo/${{ github.repository_owner
}}_${{ github.event.repository.name }}/branch/${{ github.ref_name }}
          # HELP ci_duration_seconds Duration of CI test job
          # TYPE ci_duration_seconds gauge
          ci_duration_seconds{workflow="${{ github.workflow }}"}
${DURATION}
          # HELP ci_success CI job success (1) or failure (0)
          # TYPE ci_success gauge
          ci_success{workflow="${{ github.workflow }}"} ${STATUS_VALUE}
          EOF
```

Slack/PagerDuty Notifications from Workflows

```
      # ── Slack notification on failure ──────────────────────────────
      - name: Notify Slack on failure
        if: failure() && github.ref == 'refs/heads/main'
        uses: slackapi/slack-github-
action@485a9d42d3a73031f12ec201c457e2162c45d02d
        with:
          payload: |
            {
              "text": "❌ CI Failed on main",
              "blocks": [
                {
                  "type": "section",
                  "text": {
                    "type": "mrkdwn",
                    "text": "*❌ CI Failed on `main`*\n<${{
github.server_url }}/${{ github.repository }}/actions/runs/${{
github.run_id }}|View Run>"
                  }
                },
                {
                  "type": "section",
                  "fields": [
                    {"type": "mrkdwn", "text": "*Repo:*\n${{
github.repository }}"},
                    {"type": "mrkdwn", "text": "*Commit:*\n`${{ github.sha
}}`"},
                    {"type": "mrkdwn", "text": "*Author:*\n${{
github.actor }}"},
                    {"type": "mrkdwn", "text": "*Branch:*\n${{
github.ref_name }}"}
                  ]
                }
              ]
            }
        env:
          SLACK_WEBHOOK_URL: ${{ secrets.SLACK_WEBHOOK_URL }}
          SLACK_WEBHOOK_TYPE: INCOMING_WEBHOOK

      # ── PagerDuty alert for production deploy failures ─────────────
      - name: PagerDuty alert
        if: failure() && github.ref == 'refs/heads/main'
        run: |
          curl -s -X POST https://events.pagerduty.com/v2/enqueue \
            -H "Content-Type: application/json" \
            -d '{
              "routing_key": "${{ secrets.PAGERDUTY_INTEGRATION_KEY }}",
              "event_action": "trigger",
              "payload": {
                "summary": "Production deployment failed: ${{
github.repository }}",
```

```
                  "severity": "critical",
                  "source": "github-actions",
                  "custom_details": {
                    "run_url": "${{ github.server_url }}/${{
github.repository }}/actions/runs/${{ github.run_id }}",
                    "commit": "${{ github.sha }}",
                    "actor": "${{ github.actor }}"
                  }
                }
              }'
```

Interview Answer (30-45 seconds)

> "Status badges are SVGs from `github.com/<owner>/<repo>/actions/workflows/<file>.yml/badge.svg` — they reflect the last completed run on the default branch and can be scoped to specific branches or event types via query parameters. For real observability, badges are too coarse and too cached. The more useful surfaces are: the Actions tab with filter queries for failure patterns, individual run timing breakdowns to spot slow steps and runner queue depth, and the Deployments API for environment promotion history. For team-level CI health metrics, the pattern is to emit metrics from within workflows to Datadog or Prometheus Pushgateway using the API — capturing duration, pass/fail counts, queue time. Slack notifications via `slackapi/slack-github-action` on `if: failure() && github.ref == 'refs/heads/main'` ensures the on-call team knows immediately when the main branch breaks."

Gotchas ⚠

- **Badge URLs require the exact workflow filename.** If your workflow file is `ci-pipeline.yml`, the badge URL must use `ci-pipeline.yml`, not the workflow's `name:` field. Renaming a workflow file breaks existing badge URLs.
- **Badges for private repos require authentication.** The badge SVG URL for a private repo returns a 404 to unauthenticated requests. Embedding in a public README doesn't work. Use GitHub's repository visibility badge service or a proxy with auth.
- **Badge status is based on the last COMPLETED run, not the latest in-progress run.** If a new push triggers a run that's currently executing, the badge still shows the previous run's result. "Passing" badge during a failing in-progress run is normal.
- **The Actions tab only retains runs for 90 days by default.** Workflow runs older than the retention period (configurable 1-400 days) are deleted. For long-term trend analysis, export metrics to an external system — don't rely on GitHub's retention.

9.4 GitHub Actions API — Querying Runs, Triggering via API/CLI

What It Is

The GitHub REST API exposes full control over GitHub Actions — listing and querying workflow runs, triggering workflows programmatically, cancelling runs, downloading artifacts, managing secrets, and inspecting runner status. The GitHub CLI (`gh`) wraps these APIs for interactive use.

Querying Workflow Runs

```
# ── List recent runs for a workflow ──────────────────────────
gh run list \
  --repo my-org/my-repo \
  --workflow ci.yml \
  --limit 20

# Filter by status, branch, actor
gh run list \
  --repo my-org/my-repo \
  --workflow ci.yml \
  --branch main \
  --status failure \
  --limit 10

# JSON output for scripting
gh run list \
  --repo my-org/my-repo \
  --workflow ci.yml \
  --json
databaseId,status,conclusion,createdAt,updatedAt,headBranch,headSha,name \
  --limit 50

# ── Get details of a specific run ────────────────────────────
gh run view 1234567890 \
  --repo my-org/my-repo

# View run in browser
gh run view 1234567890 --web

# ── Watch a live run ─────────────────────────────────────────
gh run watch 1234567890

# ── API equivalent (JSON) ────────────────────────────────────
gh api \
  "/repos/my-org/my-repo/actions/runs" \
  --jq '.workflow_runs[] | {id: .id, status: .status, conclusion:
.conclusion, branch: .head_branch}'

# Filter runs by workflow name AND status
gh api \
  "/repos/my-org/my-repo/actions/runs?
workflow_id=ci.yml&status=failure&branch=main" \
  --jq '[.workflow_runs[] | {id: .id, created_at: .created_at, head_sha:
```

```
.head_sha}]'

# Get a specific run's jobs
gh api \
  "/repos/my-org/my-repo/actions/runs/1234567890/jobs" \
  --jq '.jobs[] | {name: .name, status: .status, conclusion: .conclusion,
started_at: .started_at, completed_at: .completed_at}'
```

Triggering Workflows via API and CLI

```
# ── workflow_dispatch via CLI ──────────────────────────────────
# Trigger on default branch
gh workflow run ci.yml --repo my-org/my-repo

# Trigger on specific branch with inputs
gh workflow run deploy.yml \
  --repo my-org/my-repo \
  --ref release/2.0 \
  -f environment=production \
  -f dry_run=false \
  -f service=payment-api

# ── workflow_dispatch via REST API ─────────────────────────────
curl -X POST \
  -H "Authorization: Bearer $GH_TOKEN" \
  -H "Accept: application/vnd.github.v3+json" \
  "https://api.github.com/repos/my-org/my-
repo/actions/workflows/deploy.yml/dispatches" \
  -d '{
    "ref": "main",
    "inputs": {
      "environment": "production",
      "dry_run": "false",
      "service": "payment-api"
    }
  }'

# ── repository_dispatch ────────────────────────────────────────
# Requires: PAT with repo scope (NOT GITHUB_TOKEN)
curl -X POST \
  -H "Authorization: Bearer $GH_PAT" \
  -H "Accept: application/vnd.github.v3+json" \
  "https://api.github.com/repos/my-org/my-repo/dispatches" \
  -d '{
    "event_type": "deploy-service",
    "client_payload": {
      "service": "payment-api",
      "image_tag": "sha-abc123",
      "triggered_by": "orchestrator"
    }
```

```
  }'

# ── Trigger and wait for completion ──────────────────────────────
trigger_and_wait() {
  local REPO="$1"
  local WORKFLOW="$2"
  local REF="${3:-main}"
  shift 3
  local INPUTS=("$@")

  # Record time before trigger
  BEFORE=$(date -u +%Y-%m-%dT%H:%M:%SZ)

  # Trigger the workflow
  gh workflow run "$WORKFLOW" --repo "$REPO" --ref "$REF" "${INPUTS[@]}"

  # Wait for the run to appear (API has slight delay after dispatch)
  sleep 5

  # Find the run that was just created
  RUN_ID=$(gh run list \
    --repo "$REPO" \
    --workflow "$WORKFLOW" \
    --branch "$REF" \
    --json databaseId,createdAt \
    --jq "[.[] | select(.createdAt > \"$BEFORE\")] | .[0].databaseId")

  if [ -z "$RUN_ID" ] || [ "$RUN_ID" = "null" ]; then
    echo "Error: Could not find triggered run"
    return 1
  fi

  echo "Triggered run ID: $RUN_ID"

  # Watch until complete
  gh run watch "$RUN_ID" --repo "$REPO"

  # Check final status
  CONCLUSION=$(gh run view "$RUN_ID" \
    --repo "$REPO" \
    --json conclusion \
    --jq '.conclusion')

  echo "Run completed with conclusion: $CONCLUSION"
  [ "$CONCLUSION" = "success" ]
}

# Usage
trigger_and_wait my-org/my-repo deploy.yml main \
  -f environment=staging \
  -f service=auth-api
```

Managing Runs — Cancel, Rerun, Delete

```
# ── Cancel a running workflow ──────────────────────────────────
gh run cancel 1234567890 --repo my-org/my-repo

# Cancel ALL in-progress runs for a workflow on a branch
gh run list \
  --repo my-org/my-repo \
  --workflow ci.yml \
  --branch feature/my-branch \
  --status in_progress \
  --json databaseId \
  --jq '.[].databaseId' | \
  xargs -I{} gh run cancel {} --repo my-org/my-repo

# ── Re-run a failed workflow ───────────────────────────────────
gh run rerun 1234567890 --repo my-org/my-repo

# Re-run only failed jobs (not entire run)
gh run rerun 1234567890 --repo my-org/my-repo --failed-only

# Re-run with debug logging enabled
gh run rerun 1234567890 --repo my-org/my-repo --debug

# ── Delete old runs (cleanup) ──────────────────────────────────
# Delete all runs older than 30 days for a workflow
gh api "/repos/my-org/my-repo/actions/workflows/ci.yml/runs" \
  --paginate \
  --jq '.workflow_runs[] | select(.created_at < (now - 2592000 |
strftime("%Y-%m-%dT%H:%M:%SZ"))) | .id' | \
  xargs -I{} gh api --method DELETE "/repos/my-org/my-
repo/actions/runs/{}"
```

Secrets and Variables Management via API

```
# ── List secrets (names only -- values never returned) ──────────
gh secret list --repo my-org/my-repo
gh secret list --org my-org

# ── Set a secret ────────────────────────────────────────────────
gh secret set MY_SECRET --body "secret-value" --repo my-org/my-repo
echo "secret-value" | gh secret set MY_SECRET --repo my-org/my-repo

# Set org-level secret with repo visibility
gh secret set SHARED_SECRET \
  --body "value" \
  --org my-org \
  --visibility selected \
  --repos "repo1,repo2,repo3"
```

```
# ── Set a variable (non-secret, visible in logs) ──────────────────
gh variable set MY_VAR --body "non-secret-value" --repo my-org/my-repo
gh variable list --repo my-org/my-repo

# ── Bulk secret management from file ──────────────────────────────
# .env.secrets:
# DATABASE_URL=postgres://...
# API_KEY=abc123
while IFS='=' read -r KEY VALUE; do
  [ -z "$KEY" ] || [[ "$KEY" == \#* ]] && continue
  gh secret set "$KEY" --body "$VALUE" --repo my-org/my-repo
  echo "Set secret: $KEY"
done < .env.secrets
```

Workflow Usage and Billing via API

```
# ── Get billing summary ───────────────────────────────────────────
gh api /repos/my-org/my-repo/actions/billing \
  --jq '{
    total_minutes: .total_minutes_used,
    paid_minutes: .total_paid_minutes_used,
    included_minutes: .included_minutes,
    breakdown: .minutes_used_breakdown
  }'

# Org-level billing
gh api /orgs/my-org/settings/billing/actions \
  --jq '{
    total_minutes: .total_minutes_used,
    paid_minutes: .total_paid_minutes_used,
    linux_minutes: .minutes_used_breakdown.UBUNTU,
    macos_minutes: .minutes_used_breakdown.MACOS,
    windows_minutes: .minutes_used_breakdown.WINDOWS
  }'

# ── Self-hosted runner status ─────────────────────────────────────
gh api /orgs/my-org/actions/runners \
  --jq '.runners[] | {name: .name, status: .status, busy: .busy, labels:
[.labels[].name]}'

# Find idle vs busy runners
gh api /orgs/my-org/actions/runners \
  --jq '[.runners[] | select(.busy == false) | .name]'
```

Artifacts via API

```
# ── List artifacts for a run ──────────────────────────────────
gh api /repos/my-org/my-repo/actions/runs/1234567890/artifacts \
  --jq '.artifacts[] | {id: .id, name: .name, size_mb: (.size_in_bytes /
1048576 | round), created_at: .created_at}'

# ── Download artifact ─────────────────────────────────────────
gh run download 1234567890 \
  --repo my-org/my-repo \
  --name "test-results" \
  --dir ./downloaded-artifacts/

# Download all artifacts from a run
gh run download 1234567890 --repo my-org/my-repo

# ── Clean up old artifacts ────────────────────────────────────
# List all artifacts older than 7 days and delete them
gh api /repos/my-org/my-repo/actions/artifacts \
  --paginate \
  --jq '.artifacts[] | select(.created_at < (now - 604800 | strftime("%Y-
%m-%dT%H:%M:%SZ"))) | .id' | \
  xargs -I{} gh api --method DELETE /repos/my-org/my-
repo/actions/artifacts/{}
```

Interview Answer (30-45 seconds)

> "The GitHub Actions REST API and CLI expose full programmatic control over workflows. For triggering: `gh workflow run` or a POST to `/repos/.../actions/workflows/.../dispatches` for `workflow_dispatch`, and a POST to `/repos/.../dispatches` for `repository_dispatch` (the latter requires a PAT, not GITHUB_TOKEN). For querying: `gh run list` with `--status`, `--branch`, `--workflow` filters, or the API with `--jq` expressions for scripting. The trigger-and-wait pattern is important for orchestration: trigger a workflow dispatch, wait 5 seconds for the API to register the run, query by creation time to find the run ID, then `gh run watch` to block until completion and check `.conclusion`. Secrets are managed via `gh secret set` — values are encrypted client-side before upload, the API never returns secret values, only names."

Gotchas ⚠

- **There's a 2-5 second delay between `workflow_dispatch` and the run appearing in the API.** Querying immediately after triggering returns no results. Always add a `sleep 5` before querying for the newly triggered run.
- **`workflow_dispatch` from the API uses the WORKFLOW FILE on the specified `ref`,** not the default branch. If you trigger on `ref: main` but the `on.workflow_dispatch:` trigger is only on `feature/my-branch`, the dispatch silently succeeds but no run starts.
- **`gh run list` with `--json` returns runs in REVERSE chronological order** (newest first). Use `.[0]` for the most recent, not `.[-1]`.

- **`GITHUB_TOKEN` cannot trigger `repository_dispatch`.** It can trigger `workflow_dispatch`. For `repository_dispatch` to other repos, you need a PAT with `repo` scope or a GitHub App token.
- **The REST API returns at most 100 items per page.** For repos with many runs, use `--paginate` in the CLI or implement pagination with `Link` header parsing in raw API calls.
- **Deleting workflow runs doesn't delete associated artifacts.** Artifacts must be deleted separately. A run being deleted also doesn't affect the run being counted in billing — billing is captured at completion.

9.5 Rate Limits, Usage Limits, and Billing Visibility

What It Is

GitHub Actions operates within multiple constraint systems: API rate limits (how many API calls you can make), concurrent job limits (how many jobs run simultaneously), storage limits (artifacts and cache), and billing limits (how many compute minutes you consume). Understanding these prevents unexpected failures and cost surprises.

API Rate Limits

```
GitHub REST API Rate Limits:
  Unauthenticated:     60 requests / hour per IP
  Authenticated PAT:   5,000 requests / hour per user
  GITHUB_TOKEN:        1,000 requests / hour per repository
  GitHub App:          5,000 requests / hour per installation
                        (15,000 for Enterprise Cloud orgs)

Secondary rate limits (protection against abuse):
  Max concurrent requests:    ~100 requests in flight simultaneously
  Max requests per minute:    ~900 requests/minute (undocumented)
  Max requests per second:    ~100 requests/second burst (undocumented)
  Content creation rate:      ~80 events/minute for issue/PR events

Rate limit headers in every response:
  X-RateLimit-Limit:     Total calls allowed
  X-RateLimit-Remaining: Calls remaining in window
  X-RateLimit-Reset:     Unix timestamp when limit resets
  X-RateLimit-Used:      Calls used so far
```

```
# Check your current rate limit status
gh api rate_limit --jq '{
  core: {
    limit: .resources.core.limit,
    remaining: .resources.core.remaining,
    reset_in_minutes: ((.resources.core.reset - now) / 60 | round)
```

```
  },
  graphql: {
    limit: .resources.graphql.limit,
    remaining: .resources.graphql.remaining
  }
}'

# In a workflow script -- rate limit aware loop
check_and_wait_rate_limit() {
  local REMAINING=$(gh api rate_limit --jq '.resources.core.remaining')
  local RESET=$(gh api rate_limit --jq '.resources.core.reset')

  if [ "$REMAINING" -lt 100 ]; then
    local NOW=$(date +%s)
    local WAIT=$((RESET - NOW + 10))
    echo "Rate limit low ($REMAINING remaining). Waiting ${WAIT}s until
reset..."
    sleep $WAIT
  fi
}

# Use in loops
for REPO in $(cat repos.txt); do
  check_and_wait_rate_limit
  gh api /repos/$REPO/actions/runs --jq '...' >> results.json
done
```

Concurrent Job Limits

```
GitHub-hosted concurrent job limits by plan:
  Free:          20 concurrent jobs (5 for macOS)
  Team:          60 concurrent jobs (5 for macOS)
  Enterprise:   500 concurrent jobs (50 for macOS)

  GitHub Teams with larger runners (4+ cores):
    Count of concurrent larger runner jobs is separate from standard

Self-hosted:
  No concurrent limit enforced by GitHub
  Your infrastructure is the limit
  ARC maxRunners setting is your soft limit

What happens at the limit:
  New jobs are QUEUED (not failed)
  Jobs wait in queue until a slot opens
  Queued time appears in job timing
  Long queue times = you need more capacity or fewer concurrent jobs
```

```yaml
# See 9.2 category5 -- concurrency controls limit your own parallelism
# Use concurrency: at workflow or job level to stay within limits
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

# max-parallel on matrix limits how many jobs run simultaneously
strategy:
  matrix:
    service: [a, b, c, d, e, f, g, h, i, j]  # 10 combinations
  max-parallel: 5  # Only 5 run at once -- respects your plan limit
```

Storage Limits

```
GitHub Actions Cache:
  Per-repo limit:     10 GB
  TTL:                7 days since last access
  Eviction:           LRU when limit reached (oldest least-used evicted)

Artifacts:
  Per-artifact:       No per-artifact limit
  Per-run:            No per-run limit
  Per-repo:           No hard storage limit (but incurs costs)
  Retention:          1-400 days (default 90 days, configurable)
  Included storage (before billing):
    Free:         500 MB
    Pro:          1 GB
    Team:         2 GB
    Enterprise:   50 GB

GitHub Packages (GHCR):
  Included storage:
    Free (public):   Unlimited for public images
    Free (private):  500 MB
    Pro:             2 GB
    Team:            2 GB
    Enterprise:      50 GB
```

Billing Visibility and Cost Monitoring

```yaml
# ── Current billing cycle usage ──────────────────────────────────
gh api /orgs/my-org/settings/billing/actions | jq '{
  total_minutes_used: .total_minutes_used,
  paid_minutes: .total_paid_minutes_used,
  included_minutes: .included_minutes,
  estimated_cost_usd: (.total_paid_minutes_used * 0.008),
```

```
  breakdown: .minutes_used_breakdown
}'
# Note: breakdown shows UBUNTU, MACOS, WINDOWS separately
# Multiply by multiplier: UBUNTU=1x, WINDOWS=2x, MACOS=10x

# ── Storage billing ──────────────────────────────────────────
gh api /orgs/my-org/settings/billing/packages | jq '{
  total_gb: (.total_gigabytes_bandwidth_used),
  paid_gb: (.total_paid_gigabytes_bandwidth_used)
}'

gh api /orgs/my-org/settings/billing/shared-storage | jq '{
  used_gb: (.estimated_storage_for_month),
  paid_gb: (.estimated_paid_storage_for_month)
}'

# ── Per-repo breakdown (requires org admin) ──────────────────
# List all repos sorted by Actions minutes (approximate via runs)
gh api /orgs/my-org/repos --paginate \
  --jq '.[].name' | \
  while read REPO; do
    MINS=$(gh api /repos/my-org/$REPO/actions/billing \
      --jq '.total_minutes_used' 2>/dev/null || echo 0)
    echo "$MINS $REPO"
  done | sort -rn | head -20
```

Cost Monitoring Workflow (Weekly Report)

```
name: Weekly CI Cost Report

on:
  schedule:
    - cron: '0 9 * * 1'    # 9 AM UTC every Monday
  workflow_dispatch:

permissions:
  contents: read

jobs:
  cost-report:
    runs-on: ubuntu-latest
    steps:
      - name: Fetch billing data
        id: billing
        run: |
          # Org billing
          BILLING=$(gh api /orgs/my-org/settings/billing/actions)

          TOTAL=$(echo "$BILLING" | jq -r '.total_minutes_used')
          PAID=$(echo "$BILLING" | jq -r '.total_paid_minutes_used')
```

```
          UBUNTU=$(echo "$BILLING" | jq -r '.minutes_used_breakdown.UBUNTU
// 0')
          MACOS=$(echo "$BILLING" | jq -r '.minutes_used_breakdown.MACOS
// 0')
          WINDOWS=$(echo "$BILLING" | jq -r
'.minutes_used_breakdown.WINDOWS // 0')

          # Estimated cost (approximate -- actual billing varies)
          UBUNTU_COST=$(echo "scale=2; $UBUNTU * 0.008" | bc)
          MACOS_COST=$(echo "scale=2; $MACOS * 0.08" | bc)
          WINDOWS_COST=$(echo "scale=2; $WINDOWS * 0.016" | bc)
          TOTAL_COST=$(echo "scale=2; $UBUNTU_COST + $MACOS_COST +
$WINDOWS_COST" | bc)

          cat >> $GITHUB_STEP_SUMMARY << EOF
          ## Weekly GitHub Actions Cost Report

          **Billing Period:** Current month to date

          ### Compute Minutes

          | Runner | Minutes Used | Estimated Cost |
          |--------|-------------|----------------|
          | Linux (ubuntu) | $UBUNTU | \$$UBUNTU_COST |
          | macOS | $MACOS | \$$MACOS_COST |
          | Windows | $WINDOWS | \$$WINDOWS_COST |
          | **Total** | **$TOTAL** | **\$$TOTAL_COST** |

          > Note: macOS costs 10x Linux; Windows costs 2x Linux
          EOF

          echo "total=$TOTAL" >> $GITHUB_OUTPUT
          echo "cost=$TOTAL_COST" >> $GITHUB_OUTPUT
        env:
          GH_TOKEN: ${{ secrets.ORG_BILLING_TOKEN }}

      - name: Send to Slack
        run: |
          curl -X POST "${{ secrets.SLACK_WEBHOOK_URL }}" \
            -H 'Content-Type: application/json' \
            -d "{
              \"text\": \"📊 Weekly Actions Cost: \$${{
steps.billing.outputs.cost }} (~${{ steps.billing.outputs.total }} compute
minutes this month)\"
            }"
```

Interview Answer (30-45 seconds)

> "GitHub Actions has three constraint systems to understand. First, API rate limits: GITHUB_TOKEN gets 1,000 requests/hour per repo, PATs get 5,000/hour per user, GitHub Apps get 15,000/hour for Enterprise. Workflows that call the API in loops need to check `X-RateLimit-Remaining` headers

> and back off. Second, concurrent job limits: Free plan allows 20 concurrent jobs, Enterprise allows 500. Hitting the limit queues jobs rather than failing them — you see this as long 'queue time' in the job timing view. Use `max-parallel` on matrix and `concurrency:` controls to stay within your plan's limits and avoid queue spikes. Third, storage: Actions cache has a 10GB per-repo limit with 7-day LRU eviction; artifacts bill beyond the plan's included storage. The billing API at `/orgs/<org>/settings/billing/actions` gives the current month's minute breakdown by runner type — important because macOS minutes cost 10x Linux."

Gotchas ⚠

- **GITHUB_TOKEN rate limit is per REPOSITORY, not per organization.** A workflow in `my-org/repo-a` and one in `my-org/repo-b` have separate 1,000 req/hour limits. This is usually fine but matters for high-volume multi-repo automation.
- **Secondary rate limits are not well-documented and not reflected in the rate limit headers.** If you get HTTP 403 with message "secondary rate limit," you're sending too many requests too fast — add `sleep 1` between API calls, even if `X-RateLimit-Remaining` shows capacity.
- **Cache eviction is opaque.** You don't get notified when a cache entry is evicted. The workflow just misses the cache and runs longer. Monitor the cache hit rate by checking `steps.<id>.outputs.cache-hit` and tracking it over time.
- **Artifact storage billing is based on daily snapshots**, not per-artifact. GitHub calculates average storage used per day and bills the overage monthly. Deleting old artifacts reduces next month's bill but doesn't instantly reduce the current snapshot.
- **The billing API lags by up to 48 hours.** Minutes used in the last 48 hours may not be reflected in the API response. Don't use it for real-time cost alerting — it's best for weekly/monthly reporting.

9.6 Audit Logs for GitHub Actions at Org Level

What It Is

GitHub maintains audit logs for all GitHub Actions-related events at the organization level: workflow runs triggered, secrets accessed, runner registrations, permission changes, and policy modifications. These logs are essential for security compliance, incident investigation, and governance.

Jenkins equivalent: Jenkins Audit Trail plugin, which logs build triggers and configuration changes. The difference: GitHub's audit log is comprehensive and covers the entire organization — who ran what workflow, when, what secrets were accessed, what runner processed the job — all in a queryable API.

Audit Log Event Categories for GitHub Actions

```
Actions-related audit log event categories:

workflows:
  workflows.completed_workflow_run     -- Run finished (success/failure)
```

```
    workflows.created_workflow_run       -- New run initiated
    workflows.deleted                    -- Workflow file deleted
    workflows.prepared_workflow_job      -- Job dispatched to runner

  secrets:
    secret.accessed                      -- Secret read by a workflow
    secret.created                       -- New secret created
    secret.updated                       -- Secret value updated
    secret.deleted                       -- Secret removed
    secret.destroyed                     -- Secret permanently deleted
    org_secret.accessed                  -- Org-level secret accessed

  runners:
    self_hosted_runner.register          -- New self-hosted runner
  registered
    self_hosted_runner.remove            -- Runner unregistered
    self_hosted_runner_group.created     -- New runner group created
    self_hosted_runner_group.updated     -- Runner group modified
    self_hosted_runner_group.added_to_runner  -- Runner added to group

  permissions:
    org.actions_permission_change        -- Org Actions policy changed
    repo.actions_permission_change       -- Repo Actions policy changed
    actions.required_workflow            -- Required workflow added/changed
```

Querying Audit Logs via API

```
# ── Basic audit log query ──────────────────────────────────────
gh api \
  "/orgs/my-org/audit-log" \
  --jq '.[] | {action: .action, actor: .actor, created_at: .created_at,
repo: .repo}'

# ── Filter by action type ──────────────────────────────────────
# Workflow runs only
gh api \
  "/orgs/my-org/audit-log?phrase=action:workflows.created_workflow_run" \
  --paginate \
  --jq '.[] | {
    action: .action,
    actor: .actor,
    repo: .repo,
    workflow: .workflow,
    created_at: .created_at
  }'

# Secret access events
gh api \
  "/orgs/my-org/audit-log?phrase=action:secret.accessed" \
  --paginate \
```

```
  --jq '.[] | {
    action: .action,
    actor: .actor,
    repo: .repo,
    secret_name: .name,
    created_at: .created_at
  }'

# Runner registration events
gh api \
  "/orgs/my-org/audit-log?phrase=action:self_hosted_runner.register" \
  --paginate \
  --jq '.[] | {
    action: .action,
    actor: .actor,
    runner_name: .runner_name,
    runner_group: .runner_group_name,
    created_at: .created_at
  }'

# ── Time-bounded queries ──────────────────────────────────────
# Events in last 24 hours
SINCE=$(date -d '24 hours ago' -u +%Y-%m-%dT%H:%M:%SZ 2>/dev/null || \
        date -v-24H -u +%Y-%m-%dT%H:%M:%SZ)

gh api \
  "/orgs/my-org/audit-log?
phrase=created:>${SINCE}&action:workflows.completed_workflow_run" \
  --paginate \
  --jq '.'

# ── Specific actor query (who ran workflows) ──────────────────
gh api \
  "/orgs/my-org/audit-log?phrase=actor:suspicious-
user+action:workflows.created_workflow_run" \
  --paginate \
  --jq '[.[] | {repo: .repo, workflow: .workflow, created_at:
.created_at}]'
```

Audit Log Streaming (Enterprise)

```
# GitHub Enterprise Cloud can stream audit logs to:
# - Amazon S3
# - Azure Blob Storage
# - Google Cloud Storage
# - Datadog
# - Splunk

# Setup via org settings -> Audit log -> Log streaming
# Once configured, events arrive within seconds
```

```
# Splunk integration receives events like:
# {
#   "action": "workflows.created_workflow_run",
#   "actor": "dev-user",
#   "actor_ip": "1.2.3.4",
#   "actor_location": {"country_code": "US"},
#   "at": "2024-03-15T10:23:45.678Z",
#   "created_at": 1710495825678,
#   "org": "my-org",
#   "repo": "my-org/payment-service",
#   "user_agent": "...",
#   "workflow_run_id": 1234567890,
#   "workflow_id": 98765,
#   "_document_id": "abc123"
# }
```

Security-Focused Audit Queries

```
# ── Find unauthorized secret access ──────────────────────────────
# Secrets accessed by workflows NOT in your approved list

APPROVED_REPOS=("my-org/payment-api" "my-org/auth-service")

gh api "/orgs/my-org/audit-log?phrase=action:secret.accessed" \
  --paginate \
  --jq '.[] | {repo: .repo, secret: .name, actor: .actor, time:
.created_at}' | \
  while IFS= read -r LINE; do
    REPO=$(echo "$LINE" | jq -r '.repo')
    if [[ ! " ${APPROVED_REPOS[@]} " =~ " ${REPO} " ]]; then
      echo "UNEXPECTED SECRET ACCESS: $LINE"
    fi
  done

# ── Detect new runner registrations ──────────────────────────────
# Alert on any new runner registered in last hour
HOUR_AGO=$(date -u -d '1 hour ago' +%Y-%m-%dT%H:%M:%SZ 2>/dev/null || \
           date -u -v-1H +%Y-%m-%dT%H:%M:%SZ)

NEW_RUNNERS=$(gh api \
  "/orgs/my-org/audit-log?phrase=action:self_hosted_runner.register" \
  --jq "[.[] | select(.created_at > \"$HOUR_AGO\")]")

if [ "$(echo "$NEW_RUNNERS" | jq 'length')" -gt 0 ]; then
  echo "ALERT: New runners registered in last hour:"
  echo "$NEW_RUNNERS" | jq '.[] | {runner: .runner_name, actor: .actor,
time: .created_at}'
fi
```

```
# ── Permission change monitoring ──────────────────────────────
gh api \
  "/orgs/my-org/audit-log?phrase=action:org.actions_permission_change" \
  --jq '.[] | "Permission changed by \(.actor) at \(.created_at): \
(.permission)"'

# ── Workflows run from outside the org ────────────────────────
# Find runs triggered by actors who are not org members
ORG_MEMBERS=$(gh org member list my-org --json login --jq '[.[].login]')
gh api "/orgs/my-org/audit-log?
phrase=action:workflows.created_workflow_run" \
  --paginate \
  --jq '.[] | select(.actor != null) | {actor: .actor, repo: .repo, time:
.created_at}' | \
  while IFS= read -r LINE; do
    ACTOR=$(echo "$LINE" | jq -r '.actor')
    if echo "$ORG_MEMBERS" | jq -e --arg a "$ACTOR" '.[] | select(. ==
$a)' > /dev/null 2>&1; then
      : # Member -- OK
    else
      echo "EXTERNAL ACTOR triggered workflow: $LINE"
    fi
  done
```

Compliance Reporting Workflow

```
name: Weekly Security Audit Report

on:
  schedule:
    - cron: '0 8 * * 1'    # 8 AM Monday
  workflow_dispatch:

permissions:
  contents: read

jobs:
  audit-report:
    runs-on: ubuntu-latest
    steps:
      - name: Generate audit report
        run: |
          WEEK_AGO=$(date -u -d '7 days ago' +%Y-%m-%dT%H:%M:%SZ)

          # Fetch relevant events
          SECRET_ACCESSES=$(gh api \
            "/orgs/my-org/audit-log?
phrase=action:secret.accessed&after=${WEEK_AGO}" \
            --paginate --jq 'length')
```

```
          SECRET_CHANGES=$(gh api \
            "/orgs/my-org/audit-log?
phrase=action:secret.created+OR+action:secret.updated+OR+action:secret.del
eted&after=${WEEK_AGO}" \
            --paginate --jq 'length')

          RUNNER_CHANGES=$(gh api \
            "/orgs/my-org/audit-log?
phrase=action:self_hosted_runner.register+OR+action:self_hosted_runner.rem
ove&after=${WEEK_AGO}" \
            --paginate --jq 'length')

          PERMISSION_CHANGES=$(gh api \
            "/orgs/my-org/audit-log?
phrase=action:org.actions_permission_change+OR+action:repo.actions_permiss
ion_change&after=${WEEK_AGO}" \
            --paginate --jq 'length')

          TOTAL_RUNS=$(gh api \
            "/orgs/my-org/audit-log?
phrase=action:workflows.created_workflow_run&after=${WEEK_AGO}" \
            --paginate --jq 'length')

          cat >> $GITHUB_STEP_SUMMARY << EOF
          ## Weekly GitHub Actions Security Audit

          **Period:** Last 7 days

          ### Activity Summary

          | Category | Count |
          |----------|-------|
          | Workflow runs initiated | $TOTAL_RUNS |
          | Secret accesses | $SECRET_ACCESSES |
          | Secret modifications (create/update/delete) | $SECRET_CHANGES
|
          | Runner registration changes | $RUNNER_CHANGES |
          | Permission policy changes | $PERMISSION_CHANGES |

          ### Action Items
          $(if [ "$PERMISSION_CHANGES" -gt 0 ]; then
            echo "- ⚠ **$PERMISSION_CHANGES permission changes** - review
in audit log"
          fi)
          $(if [ "$RUNNER_CHANGES" -gt 0 ]; then
            echo "- 🔍 **$RUNNER_CHANGES runner changes** - verify all
registrations are authorized"
          fi)

          [View full audit log](https://github.com/organizations/my-
org/settings/audit-log?q=action:workflows)
          EOF
        env:
          GH_TOKEN: ${{ secrets.ORG_AUDIT_TOKEN }}
```

```
      - name: Send report to Slack
        run: |
          curl -X POST "${{ secrets.SLACK_WEBHOOK_URL }}" \
            -H 'Content-Type: application/json' \
            -d "{\"text\": \"Weekly Actions audit report generated — see
run summary: ${{ github.server_url }}/${{ github.repository
}}/actions/runs/${{ github.run_id }}\"}"
```

Audit Log Retention

```
Retention periods:
  Free / Pro / Team:            90 days (web UI and API)
  Enterprise (web):             180 days
  Enterprise (API):             Queryable for up to 7 years
                                (with log streaming to external storage)

Best practices for long-term retention:
  1. Enable log streaming to S3/GCS/Datadog (Enterprise)
  2. For non-Enterprise: run a weekly job that exports audit logs to S3
  3. Use Splunk/Elasticsearch for searchable long-term audit storage
  4. For compliance: store as immutable (S3 Object Lock, GCS object hold)
```

```
# Non-Enterprise: Weekly audit log export to S3
name: Audit Log Export

on:
  schedule:
    - cron: '0 2 * * *'    # Daily at 2 AM

permissions:
  id-token: write

jobs:
  export:
    runs-on: ubuntu-latest
    steps:
      - uses: aws-actions/configure-aws-
credentials@e3dd6a429d7300a6a4c196c26e071d42e0343502
        with:
          role-to-assume: ${{ secrets.AUDIT_EXPORT_ROLE_ARN }}
          aws-region: us-east-1

      - name: Export yesterday's audit log
        run: |
          DATE=$(date -u -d 'yesterday' +%Y-%m-%d 2>/dev/null || \
                date -u -v-1d +%Y-%m-%d)
          START="${DATE}T00:00:00Z"
```

```
            END="${DATE}T23:59:59Z"

            gh api \
              "/orgs/my-org/audit-log?phrase=created:${START}..${END}" \
              --paginate \
              --jq '.' > "audit-${DATE}.json"

            # Upload to S3 with immutable storage
            aws s3 cp \
              "audit-${DATE}.json" \
              "s3://my-org-audit-logs/github-actions/${DATE}/audit.json" \
              --storage-class STANDARD_IA

            EVENTS=$(jq 'length' "audit-${DATE}.json")
            echo "Exported $EVENTS audit events for $DATE"
          env:
            GH_TOKEN: ${{ secrets.ORG_AUDIT_TOKEN }}
```

Interview Answer (30-45 seconds)

> "GitHub's org-level audit log captures all significant Actions events: workflow runs created and completed, secrets accessed and modified, runner registrations and removals, and permission policy changes. The audit log API at `/orgs/<org>/audit-log` supports filtering by `action:` type, actor, repo, and time range. For security use cases, the highest-value queries are: secrets accessed by unexpected repos (detect credential misuse), new runner registrations in the last hour (detect unauthorized compute), and permission policy changes (detect privilege escalation). For compliance, GitHub Enterprise supports real-time streaming to S3, GCS, Datadog, and Splunk — events arrive within seconds. For non-Enterprise, a daily workflow that exports the audit log to S3 using the API provides defensible long-term retention. Audit log retention varies: 90 days for Team plan, 180 days for Enterprise web UI, but the API supports 7-year queries on Enterprise."

Gotchas ⚠

- **The audit log API requires an Organization Owner token**, not a regular repo PAT or GITHUB_TOKEN. Create a dedicated service account with org owner permissions, or use a GitHub App with the `audit_log:read` permission.
- **Audit log entries for `secret.accessed` show the SECRET NAME but not the value.** The values are never logged anywhere. If you need to know WHEN a secret was used in a specific workflow, correlate the `secret.accessed` event time with the `workflows.created_workflow_run` event time for the same repo.
- **The `phrase=` query parameter in the audit log API uses GitHub's search syntax**, not SQL. Combining conditions uses `+` for AND and spaces for OR. Documentation is sparse — test queries in the web UI before scripting them.
- **Audit log API is paginated and can be slow for large orgs.** Fetching all events for a 1,000-repo org for a 30-day period can return millions of records. Always bound queries by time range and action type.

- **Audit log streaming to external systems has eventual consistency.** Events may arrive out of order or with up to 30-second delay. Don't use streaming for real-time alerting with zero tolerance for delay — use webhooks for that.
- **Forked repo PRs do NOT generate `secret.accessed` audit events** for the secrets they can't access. If a fork PR workflow tries to use a secret and fails, the failure is visible in the workflow logs, not the audit log.

Cross-Topic Interview Questions

Q: A security audit asks you to prove that secret `PROD_DEPLOY_KEY` has only ever been accessed by the deploy workflow and no other workflow. How do you do this?

> "Query the audit log for all `secret.accessed` events where the secret name matches `PROD_DEPLOY_KEY`. For each event, the audit log includes the repo and actor. Cross-reference with the workflow run IDs to confirm the accessing workflow's name. In shell:
>
> ```
> gh api '/orgs/my-org/audit-log?phrase=action:secret.accessed' --paginate --
> jq '[.[] | select(.name == \"PROD_DEPLOY_KEY\")]'
> ```
>
> Each event includes `.repo`, `.actor`, and a correlation to the workflow run. If I see any repo other than `my-org/payment-service` or any workflow name other than `deploy.yml`, that's a finding. For an organization with Enterprise log streaming, pull the same data from Splunk or S3 where it's been immutably archived beyond the API's 90-day retention window. For future prevention, scope the secret to only that specific repo using the secrets API with `visibility: selected`."

Q: Your CI has been failing intermittently for 3 days. The failures are non-deterministic — same commit, same branch, sometimes passes, sometimes fails. Walk through your debugging process.

> "Systematic approach across four layers.
>
> Layer 1: Pattern detection via API. Query all runs for this workflow, filter to failures, extract the job name and step that failed. `gh run list --workflow ci.yml --status failure --json databaseId,conclusion,createdAt,headSha`. If always the same step, it's reproducible enough to debug with tmate. If random steps, it's environmental.
>
> Layer 2: Timing analysis. Check the Actions tab for queue times and step durations. Long queue time means runner exhaustion. Variable test step duration means resource contention. If the test takes 2 min sometimes and 8 min other times, something is competing for CPU/disk.
>
> Layer 3: Runner environment. Add a debug step that captures `nproc`, `free -m`, `df -h`, and `uptime` at the start of each job. Correlate failures with high load average or low memory. If using self-hosted runners, check if another team's jobs are competing.
>
> Layer 4: Interactive debug. Trigger with `workflow_dispatch` and `debug_enabled: true` to get a tmate session. Reproduce the failure interactively. Check for flaky external dependencies (test

> database, external API), timing-sensitive assertions, or port conflicts between parallel test processes."

Q: You want to implement a policy: no workflow in the org can use `pull_request_target` trigger without explicit platform team approval. How do you enforce this?

> "Three-layer enforcement. Layer 1: GitHub Actions org policy. Set the org to only allow actions from `github_owned_allowed` plus a curated allow-list — this doesn't restrict triggers directly but reduces action supply chain attack surface.
>
> Layer 2: Required workflow. On GitHub Enterprise, create a required workflow in `my-org/.github` that scans all workflow files in a PR for `pull_request_target` usage. The required workflow must pass for any PR to merge. The check script uses `grep -r 'pull_request_target' .github/workflows/` and fails if found without an approval file (e.g., `.github/pull_request_target.approved` created by platform team CODEOWNERS process).
>
> Layer 3: CODEOWNERS. Add `CODEOWNERS` entry: `.github/workflows/ @my-org/platform-team`. Any PR that modifies workflow files requires platform team review before merge. Combined with branch protection requiring CODEOWNERS approval, no workflow change bypasses review. For the audit trail, each approved `pull_request_target` usage is documented in the PR review comments and captured in the audit log."

Quick Reference Cheat Sheet

Workflow Commands

```
# Grouping
echo "::group::Section title"
echo "::endgroup::"

# Annotations (appear in PR Checks tab)
echo "::notice file=src/app.ts,line=42::Message"
echo "::warning title=Title::Message"
echo "::error file=path,line=N,endLine=N::Message"

# Debug (only visible when ACTIONS_STEP_DEBUG=true)
echo "::debug::Diagnostic message"

# Masking (BEFORE any logging of the value)
echo "::add-mask::${SECRET_VALUE}"

# Summary (rich Markdown in run Summary tab)
echo "## Header" >> $GITHUB_STEP_SUMMARY
echo "| Col1 | Col2 |" >> $GITHUB_STEP_SUMMARY

# Environment (available to subsequent steps)
echo "MY_VAR=value" >> $GITHUB_ENV
```

```
# PATH (available to subsequent steps)
echo "/path/to/tools" >> $GITHUB_PATH

# DEPRECATED (do not use)
# ::set-output::  ->  echo "key=value" >> $GITHUB_OUTPUT
# ::set-env::     ->  echo "KEY=value" >> $GITHUB_ENV
```

tmate Quick Reference

```
- uses: mxschmitt/action-tmate@v3
  if: failure()               # Debug on failure
  with:
    limit-access-to-actor: true   # Only PR author can SSH
    timeout-minutes: 15           # Auto-exit (REQUIRED in production)

# Enable via workflow_dispatch input:
- uses: mxschmitt/action-tmate@v3
  if: inputs.debug_enabled == 'true'
```

act Quick Reference

```
act                           # Run default push event
act --job build               # Run specific job
act pull_request              # Run PR event
act workflow_dispatch \       # Run with inputs
  --input env=staging
act --list                    # List available jobs
act --dryrun                  # Preview without running
act --secret-file .secrets    # Load secrets from file
```

Badge URLs

```
# Basic
![CI](https://github.com/ORG/REPO/actions/workflows/FILE.yml/badge.svg)
# Branch-specific
![CI](https://github.com/ORG/REPO/actions/workflows/FILE.yml/badge.svg?
branch=main)
# Event-specific
![CI](https://github.com/ORG/REPO/actions/workflows/FILE.yml/badge.svg?
event=push)
```

Key API Calls

```
# Trigger workflow
gh workflow run WORKFLOW.yml --ref main -f key=value

# List runs with filters
gh run list --workflow WORKFLOW.yml --status failure --branch main

# Cancel run
gh run cancel RUN_ID

# Re-run failed jobs only
gh run rerun RUN_ID --failed-only --debug

# Check rate limits
gh api rate_limit --jq '.resources.core'

# Billing
gh api /orgs/ORG/settings/billing/actions

# Audit log (requires org owner token)
gh api "/orgs/ORG/audit-log?phrase=action:secret.accessed"
```

Rate Limits Summary

```
GITHUB_TOKEN:    1,000 req/hour per repo
PAT:             5,000 req/hour per user
GitHub App:      5,000 req/hour (15,000 Enterprise)

Concurrent jobs:
  Free:          20 (5 macOS)
  Team:          60 (5 macOS)
  Enterprise:    500 (50 macOS)

Storage:
  Cache:         10 GB per repo, 7-day LRU
  Artifacts:     90-day default retention
  GHCR:          500MB free (private)
```

Audit Log Key Events

```
# Secret access
phrase=action:secret.accessed

# Workflow triggers
phrase=action:workflows.created_workflow_run

# Runner changes
phrase=action:self_hosted_runner.register
```

```
# Permission changes
phrase=action:org.actions_permission_change

# Time-bounded
phrase=action:secret.accessed+created:>2024-01-01
```

End of Category 9: Observability, Debugging & Operations

> **Next:** Category 10 (GitHub Actions vs Jenkins & Migration Patterns) synthesizes everything from Categories 1–9 into a comprehensive comparison and migration framework — the most common interview topic for engineers transitioning a Jenkins-heavy organization to GitHub Actions.
>
> **Priority drill topics from this category:**
>
> - Workflow commands: write all six from memory (`::group::`, `::error::`, `::add-mask::`, `::debug::`, `$GITHUB_STEP_SUMMARY`, `$GITHUB_ENV`) and explain the masking-before-logging requirement
> - tmate: explain the security concerns (`limit-access-to-actor`, `timeout-minutes`) and when you'd use it vs act
> - Rate limits: distinguish GITHUB_TOKEN (1k/repo) vs PAT (5k/user) vs App (15k Enterprise); explain secondary rate limit behavior
> - Audit log: name the four security-critical event types and the required permission level for audit log API access